

Developer's Guide

Backend database

The backend database is used to store the documents, questions, answers, and user information. The backend database is built using Django and PostgreSQL. The database can be accessed by creating a superuser and logging in to the admin panel.

create super user

To create super user, run following command if you are using pre-built docker images:

```
cd MyGPT/installation/macOS/prebuilt_images/  
bash create_superuser.sh
```

Or run following command if you are building docker images:

```
bash MyGPT/installation/macOS/build_images/create_superuser.sh
```

You can check backend database at <http://localhost:8000/admin/> with username and password you created in above step.

API Endpoints

These endpoints are implemented in `backend/testdb/views/apis.py` and routed in `backend/testdb/urls.py`.

Minimal frontend payloads (5 most used)

Use these examples as a quick integration reference.

1) POST `/api/get_datasets/`

Request:

```
{  
  "user_email": "abc@xyz.com",  
  "user_group": ""  
}
```

Response:

```
[  
  {  
    "dataset_name": "GPCR",  
    "dataset_size": 3366,  
    "user_email": "abc@xyz.com"  
  }  
]
```

2) POST `/api/get_documents/`

Request:

```
{
  "dataset": "GPCR",
  "user_email": "abc@xyz.com",
  "user_group": ""
}
```

Response:

```
{
  "documents": [],
  "dataset_type": "papers"
}
```

3) POST /api/get_context/

Request:

```
{
  "text": "Which cohorts are available from Survivorship portal?",
  "model_type": "llama3:latest",
  "dataset": "Survivorship",
  "new_conversation": true,
  "previous_query": "",
  "no_context": false,
  "use_default_qrs": true,
  "question_best_distance": 0.2,
  "question_worst_distance": 1.7,
  "maximum_chunks_count": 5,
  "no_cutoff": false
}
```

Response (shape):

```
{
  "context": "...",
  "relevance_score": 61,
  "semantic_score": 58,
  "keyword_score": 42,
  "rerank_score": 0,
  "sources": []
}
```

4) POST /api/save_answer/

Request:

```
{
  "question_text": "Which cohorts are available from Survivorship portal?",
}
```

```

"answer_text": "...",
"answer_no_context_text": "...",
"model_type": "llama3:latest",
"dataset": "Survivorship",
"no_context": false,
"use_default_ars": true,
"answer_best_distance": 0.12,
"answer_worst_distance": 1.42,
"QRS_p": 1,
"ARS_q": 2,
"use_default_hi": true,
"temperature": 0.4,
"top_k": 20,
"top_p": 0.7
}

```

Response (shape):

```

{
  "saved": true,
  "mean_distance_a": 0.402,
  "relevance_score": 73,
  "semantic_score": 71,
  "keyword_score": 0,
  "hallucination_index_by_equation": 22,
  "hallucination_index_by_ml": 19,
  "sources": []
}

```

5) GET /api/get_question_details/?question_id=<id>

Request example:

```
GET /api/get_question_details/?question_id=1
```

Response (shape):

```

{
  "question": "...",
  "relevance_score": 66,
  "ground_truth": "-",
  "question_type": "other",
  "keywords": "",
  "llm": "llama3:latest",
  "answers": [],
  "sources": []
}

```

Core chat APIs

1) Get datasets

- **Method/Path:** POST /api/get_datasets/
- **Body:**

```
{
  "user_email": "abc@xyz.com",
  "user_group": ""
}
```

- **Response:** array of dataset field objects (sorted by dataset_name).

```
[
  {
    "dataset_name": "GPCR",
    "dataset_size": 3366,
    "user_email": "abc@xyz.com"
  }
]
```

2) Get dataset details

- **Method/Path:** POST /api/get_dataset_details/
- **Body:**

```
{
  "dataset": "GPCR",
  "user_email": "abc@xyz.com",
  "user_group": ""
}
```

- **Response:** single dataset field object.

3) Update dataset settings

- **Method/Path:** POST /api/update_dataset/
- **Body:** dataset is required. You can send one or more updatable fields.

```
{
  "dataset": "GPCR",
  "system_prompt": "You are a biomedical assistant.",
  "Qsem_a": 4,
  "Qkey_b": -4,
  "Qrank_c": -1,
  "Asem_x": 5,
  "Akey_y": -2,
  "Arank_z": 0,
  "QRS_p": 1,
  "ARS_q": 2,
}
```

```
    "HI_by_equation": true
}
```

- **Response:**

```
{
  "saved": true,
  "dataset": {
    "dataset_name": "GPCR"
  }
}
```

4) Get documents in a dataset

- **Method/Path:** POST /api/get_documents/
- **Body:**

```
{
  "dataset": "GPCR",
  "user_email": "abc@xyz.com",
  "user_group": ""
}
```

- **Response:**

```
{
  "documents": [],
  "dataset_type": "papers"
}
```

dataset_type is papers or videos .

5) Get retrieval context (RAG)

- **Method/Path:** POST /api/get_context/
- **Body (required keys):**

```
{
  "text": "Which cohorts are available from Survivorship portal?",
  "model_type": "llama3:latest",
  "dataset": "Survivorship",
  "new_conversation": true,
  "previous_query": "",
  "no_context": false,
  "use_default_qrs": true,
  "question_best_distance": 0.2,
  "question_worst_distance": 1.7,
  "maximum_chunks_count": 5,
}
```

```

    "no_cutoff": false
}

```

- **Optional keys:** `focused_document_titles` , `focused_section` , `keywords` , `translated_text` , `skip_highlight` .
- **Response includes:** `context` , `relevance_score` , `semantic_score` , `keyword_score` , `rerank_score` , `sources` .

6) Save generated answer + metrics

- **Method/Path:** POST `/api/save_answer/`
- **Body (required keys):**

```

{
  "question_text": "Which cohorts are available from Survivorship portal?",
  "answer_text": "...",
  "answer_no_context_text": "...",
  "model_type": "llama3:latest",
  "dataset": "Survivorship",
  "no_context": false,
  "use_default_ars": true,
  "answer_best_distance": 0.12,
  "answer_worst_distance": 1.42,
  "QRS_p": 1,
  "ARS_q": 2,
  "use_default_hi": true,
  "temperature": 0.4,
  "top_k": 20,
  "top_p": 0.7
}

```

- **Optional key:** `translated_answer_text` .
- **Response (when `no_context=false`) includes:** `saved` , `mean_distance_a` , `relevance_score` , `semantic_score` , `keyword_score` , `hallucination_index_by_equation` , `hallucination_index_by_ml` , `sources` .
- **Response (when `no_context=true`):** `saved` , `relevance_score` .

7) Get question details

- **Method/Path:** GET `/api/get_question_details/?question_id=<id>`
- **Response includes:** `question` , `relevance_score` , `ground_truth` , `question_type` , `keywords` , `llm` , `answers` , `sources` .

`answers[]` fields:

- `answer`
- `relevance_score`
- `hallucination_index_by_equation`
- `hallucination_index_by_ml`
- `answer_no_context`

`sources[]` fields include `vector/BM25/rerank` values and `rank` fields.

8) Get conversation history by dataset

- **Method/Path:** GET /api/get_conversation_history/?dataset=<dataset_name>
- **Response:**

```
{
  "conversations": [
    {
      "conversation_id": 6664,
      "questions_answers": [
        {
          "question_id": 7547,
          "question": "What kind of analysis can be performed?",
          "relevance_score": 58
        }
      ]
    }
  ]
}
```

9) Save answer feedback

- **Method/Path:** POST /api/feedback/
- **Body:**

```
{
  "answer_text": "...",
  "rating": 5,
  "user_comment": "Helpful answer"
}
```

- **Response:** { "saved": true }

Dataset/library management APIs

10) Upload documents

- **Method/Path:** POST /api/upload_documents/
- **Content type:** multipart form-data
- **Behavior:** creates/updates dataset via uploaded files, optionally indexes BM25, then adds vectors to Chroma.
- **Response:** { "uploaded": true }

11) Add Zotero collection

- **Method/Path:** POST /api/add_zotero_collection/
- **Content type:** multipart form-data
- **Body fields:** api_key, library_id, library_id_type, collection_id, embedding_model, distance_function, chunking_method, use_bm25, user, user_email, user_group.

- **Response:** { "added": true, "datasets": ["..."] } or { "error": true, "error_message": "..." } .

12) Add video library

- **Method/Path:** POST /api/add_video_library/
- **Content type:** multipart form-data
- **Body fields:** dataset_name , embedding_model , video_urls (comma-separated), playlist_url , user , user_email , user_group .
- **Response:** { "added": true } .

13) Delete dataset

- **Method/Path:** GET /api/delete_dataset/?dataset=<name>&user_email=<email>
- **Response:** { "deleted": true } or error JSON.

14) Add demo library

- **Method/Path:** GET /api/add_demo_library/?embedding_model=<model>
- **Response:** { "added": true }

15) Get dataset sections

- **Method/Path:** POST /api/get_sections/
- **Body:**

```
{
  "dataset_name": "GPCR"
}
```

- **Response:** { "sections": [{ "section_title": "Methods", "section_count": 42 }] }

Embedding/analytics APIs

16) Get vector embeddings (PCA)

- **Method/Path:** GET /api/get_vector_embeddings/?datasets=<d1,d2>&question_id=<optional_id>
- **Response:** { "pca_embeddings": [...] } }

17) Add dataset embeddings/PCA

- **Method/Path:** GET /api/add_dataset_embeddings/?dataset=<dataset_name>
- **Response:** { "added": true }

18) Get embedding model details for a dataset

- **Method/Path:** GET /api/get_embedding_model_details/?dataset=<dataset_name>
- **Response:** { "embedding_model": { ...distance ranges... } }

19) Get distance between two answers

- **Method/Path:** POST /api/get_distance_between_answers/
- **Body:**

```
{
  "sentence1": "...",
  "sentence2": "...",
  "embedding_model": "multi-qa-MiniLM-L6-cos-v1"
}
```

- **Response:** { "distances": [...] }

LLM model APIs

20) Get installed Ollama models

- **Method/Path:** POST /api/get_ollama_models/
- **Response:** { "added": true, "models": [...] }

21) Register Ollama models in DB

- **Method/Path:** POST /api/add_ollama_models/
- **Body:**

```
{
  "llms": [
    { "name": "llama3:latest", "size": 4661224676 }
  ]
}
```

- **Response:** { "added": true }

22) Register embedding models in DB

- **Method/Path:** POST /api/add_embedding_models/
- **Body:**

```
{
  "embedding_models": [
    { "name": "multi-qa-MiniLM-L6-cos-v1", "size": 1, "source": "sentence-
transformers" }
  ]
}
```

- **Response:** { "added": true }

23) Ollama text generation

- **Method/Path:** POST /api/ollama_generate/
- **Supports:**
 - non-streaming JSON response (stream=false)
 - streaming NDJSON response (stream=true , content-type application/x-ndjson)
- **Body fields:** model , prompt , optional system , stream , think , options.temperature , options.top_k , options.top_p .

24) Ollama chat completion

- **Method/Path:** POST /api/ollama_chat/
- **Supports:**
 - non-streaming JSON response (stream=false)
 - streaming NDJSON response (stream=true , content-type application/x-ndjson)
- **Body fields:** model , messages , optional stream , think , and sampling options.

25) Ollama pull model (streaming progress)

- **Method/Path:** POST /api/ollama_pull_model/
- **Body:**

```
{
  "name": "llama3:latest"
}
```

- **Response:** NDJSON progress events (status , progress , layer_complete , final done=true).

Settings/auth/media APIs

26) Frontend settings

- **Method/Path:** GET /api/frontend_settings/
- **Response shape:**

```
{
  "settings": {
    "show_no_context_switch": false,
    "restriction_without_login": false,
    "azure_login": false,
    "django_login": false,
    "disable_chat_without_login": false
  }
}
```

Settings meaning:

- show_no_context_switch : allow direct chat mode toggle.
- restriction_without_login : prevent uploads without login.
- azure_login : enable Azure/Microsoft SSO.
- django_login : enable Django authentication.
- disable_chat_without_login : block chat usage without login.

27) Resolve username from access token

- **Method/Path:** POST /api/get_username/
- **Body:** { "access_token": "<jwt_access_token>" }
- **Response:** username , user_email , user_group (or empty username when invalid).

28) Submit disclaimer agreement

- **Method/Path:** POST /api/submit_disclaimer_agreement/
- **Body:** { "username": "<django_username>" }

- **Response:** { "agreed": true }

29) Secure media file access

- **Method/Path:** GET /media/<path:file_path>/
- **Behavior:** serves files from MEDIA_ROOT ; may require auth token/user login depending on frontend settings.

30) Logout (JWT refresh token blacklist)

- **Method/Path:** POST /logout/
- **Body:** { "refresh_token": "<jwt_refresh_token>" }
- **Permission:** authenticated user required.